

Memory, Resource Constraints & Reliability

Zephyr RTOS Intermediate Course





Quick Recap — Lecture 4



- Message queues (`k_msgq`): buffered copy-based IPC
- FIFOs & pipes: pointer passing and byte streams
- Backpressure: drop-newest, drop-oldest, proactive flow control
- Publish-subscribe with zbus: channels, listeners, subscribers
- Module boundaries: coupling, interface ownership



Today's Agenda



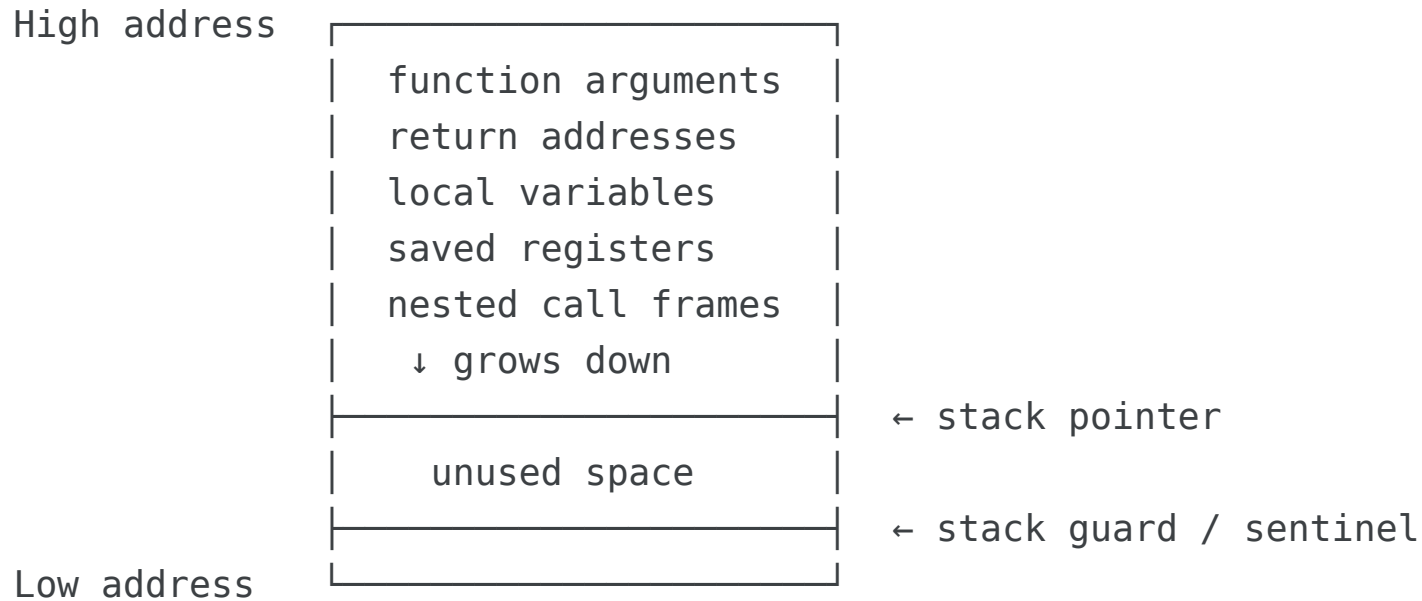
- Stack management: sizing, overflow, detection
- Heap usage: fragmentation, determinism, failure paths
- Hidden memory costs: what uses RAM beyond your code
- RAM/ROM analysis: build-time inspection tools
- Resource exhaustion patterns
- Reliability strategies: watchdogs, failure containment



Stack Management



What Lives on a Thread's Stack



- **Every function call** adds a frame
- **Every local variable** consumes stack space
- **Interrupts** use the same stack or a dedicated ISR stack



```
/* Too small will overflow under moderate load */  
K_THREAD_DEFINE(my_thread, 256, thread_fn, ...);  
/* Reasonable starting point for a thread using LOG */  
K_THREAD_DEFINE(my_thread, 1024, thread_fn, ...);  
/* Always add headroom for interrupt context saves */  
K_THREAD_DEFINE(my_thread, 2048, thread_fn, ...);
```

■ Hidden consumers that inflate stack requirements:

- Context save frames
- Logging and formatting calls
- `sprintf` / `snprintf`
- Nested function calls
- Rare error paths not hit during normal testing



Stack Overflow Detection



Choose based on hardware availability:

Method	Config	When it catches overflow
Stack sentinel	<code>CONFIG_STACK_SENTINEL</code>	At context switch, interrupt return, <code>k_yield()</code>
HW stack guard	<code>CONFIG_HW_STACK_PROTECTION</code>	Usually immediate, requires hardware support
Thread analyzer	<code>CONFIG_THREAD_ANALYZER</code>	Measures high-water mark for sizing

- **Sentinel:** software only, no hardware required, always enable in development
- **HW guard:** faster detection, more reliable, enable if target supports it
- **Thread analyzer:** run after stress test, then right-size the stack



Measuring Actual Stack Usage



```
/* CONFIG_THREAD_ANALYZER=y required */  
#include <zephyr/debug/thread_analyzer.h>  
/* Dump stack usage for all threads to the log */  
thread_analyzer_print(0);
```

Typical output:

```
THREAD ANALYZER  
my_thread      : STACK: unused 312 usage 712 / 1024 (69%)  
sysworkq       : STACK: unused 688 usage 336 / 1024 (32%)  
idle           : STACK: unused 248 usage  24 /  272  (8%)
```




Heap: Dynamic Memory



The Heap in Zephyr



```
CONFIG_HEAP_MEM_POOL_SIZE=4096    # enable the system heap (default: 0)
```

```
void *ptr = k_malloc(64);    /* allocate from system heap */  
if (ptr == NULL) {  
    /* heap exhausted – handle the failure */  
}  
k_free(ptr);
```

- Fragmentation: enough total free bytes but no contiguous block large enough
- Grows silently over time – the system works in testing, fails after hours in production



When Heap Is Acceptable



Context	Use heap?	Reason
Initialization / startup	Yes	One-time, no fragmentation risk
Configuration parsing	Yes	Short-lived, bounded
Continuous data pipeline	Avoid	Fragmentation builds over time
ISR context	Never	Dynamic allocation is not an ISR-safe design
Long-running work items	Careful	Use slab or static pool instead

Rule: if an allocation is repeated over the lifetime of the system, use a slab or static buffer instead of the heap.



Hidden Memory Costs



What	Why it's large
Threads	Full stack reserved for system lifetime. Sleeping threads still hold RAM
Queues	<code>K_MSGQ_DEFINE(q, 64, 16, 4)</code> = 1 KB buffer alone
Log buffer	<code>CONFIG_LOG_BUFFER_SIZE</code> default 1024 bytes, ring buffer always resident
Subsystems	Bluetooth, networking, TLS can add significant RAM/ROM before your code runs

```
/* This infrastructure consumes RAM before a single line of your logic */
K_THREAD_DEFINE(t1, 1024, ...); /* ~1200 bytes (struct + stack) */
K_THREAD_DEFINE(t2, 1024, ...); /* ~1200 bytes */
K_MSGQ_DEFINE(q, 64, 16, 4); /* ~1040 bytes (buffer + struct) */
```



Stack Cost of Every Thread



- **Sleeping thread is not free memory**

Every thread holds its full stack for the entire system lifetime: sleeping, waiting, or running

Memory region	Lifetime	Example
Stack	Per-thread, always resident	<code>K_THREAD_DEFINE(t, 1024, ...)</code>
Heap	Dynamic runtime	<code>k_malloc()</code> / <code>k_free()</code>
Static / global	Entire program lifetime	<code>static uint8_t rx_buf[4096]</code>

- Global buffers are as permanent as stacks and just as easy to forget in a RAM budget



RAM / ROM Analysis Tools



```
west build -t ram_report > ram.txt    # static RAM: BSS + data, by object file
west build -t rom_report > rom.txt    # flash: code + read-only data, by object file

grep -i bluetooth ram.txt            # filter by subsystem
grep -i logging rom.txt
west build -t ram_report | sort -k2 -nr | head -20  # largest consumers first
```

- **What they show:** static allocation by compiled object — which file or subsystem owns each byte
- **What they miss:** heap (runtime), stack usage (execution-dependent)
- **Practical limit:** on real projects the output is large — always redirect to a file and search



zephyr.map



- Generated automatically by every build

build/zephyr/zephyr.map

```
.bss          # all BSS symbols
.data         # initialized data
.text         # code sections
net_buf       # specific symbol
bt_           # all Bluetooth symbols
```



DEMO 1: Stack Overflow & Thread Analyzer



Resource Exhaustion Patterns



Pattern 1: Queue Buildup



producer (fast) → **[queue: depth=8]** → consumer (slow or stuck)

Time 0: [][][][][][][][] queue empty

Time 1s: [*][*][*][*][*][][][] filling

Time 2s: [*][*][*][*][*][*][*][*] full

Time 2+: producer blocks OR drops – system degrades

Early warning signal: `k_msgq_num_used_get()` trending upward over time

```
/* Health check: log queue fill level periodically */
uint32_t used = k_msgq_num_used_get(&my_q);
if (used > (QUEUE_DEPTH * 3 / 4)) {
    LOG_WRN("Queue at %d/%d", used, QUEUE_DEPTH);
}
```



Pattern 2: Heap Starvation



System boots → allocations during init → small frees → more allocs
→ fragmentation grows → ...
→ allocation for 200 bytes fails (free=600, largest contiguous=150)
→ NULL returned, not checked → crash or silent corruption

Detection strategy:

```
void *ptr = k_malloc(size);
if (ptr == NULL) {
    /* Do NOT continue. Log and fail cleanly. */
    LOG_ERR("Heap exhausted: requested %zu bytes", size);
    /* Option A: fail this operation gracefully */
    /* Option B: trigger the fatal-error path intentionally */
    k_oops();
}
```



Pattern 3: Thread Proliferation



```
"I need another task for X" → +1 thread (1200 bytes)
"I need another task for Y" → +1 thread (1200 bytes)
"I need another task for Z" → +1 thread (1200 bytes)
...
10 threads × 1200 bytes = 12000 bytes before any application data
```

Need	Thread
Long blocking I/O	Thread
Short, non-blocking processing	Work item
Periodic, non-blocking work	Work item
Priority-critical response	Thread



Reliability Strategies



Every allocation has a failure path. Design it before writing the success path.

```
/* Pattern: check, log, decide */
struct sensor_data *buf;

if (k_mem_slab_alloc(&data_slab, (void **)&buf, K_NO_WAIT) != 0) {
    /* Slab exhausted, all blocks in use */
    LOG_WRN("Data slab full, dropping sample seq=%d", seq);
    stats.dropped_samples++;
    return; /* do not continue with invalid pointer */
}
/* Success path: only reached with valid buf */
fill_data(buf);
k_fifo_put(&data_fifo, buf);
```

- Define the drop/error strategy at design time, not at debug time
- Log every drop with enough context to diagnose in the field



Hardware Watchdog



A watchdog timer resets the system if software stops responding:

```
#include <zephyr/drivers/watchdog.h>

const struct device *wdt = DEVICE_DT_GET(DT_ALIAS(watchdog0));

/* Install a 5-second timeout */
struct wdt_timeout_cfg wdt_cfg = {
    .window.max = 5000,    /* ms */
    .callback    = NULL,   /* NULL = reset immediately on timeout */
    .flags       = WDT_FLAG_RESET_SOC,
};

int chan = wdt_install_timeout(wdt, &wdt_cfg);
wdt_setup(wdt, WDT_OPT_PAUSE_HALTED_BY_DBG);

/* Feed regularly, if this stops, system resets after 5 seconds */
wdt_feed(wdt, chan);
```



Task Watchdog: Per-Thread Supervision



```
#include <zephyr/task_wdt/task_wdt.h>
task_wdt_init(NULL); /* NULL = software-only; pass hw wdt device for fallback */

/* Register a channel per monitored thread */
int chan = task_wdt_add(2000, /* timeout ms */
                       my_callback, /* called if thread misses feed */
                       (void *)k_current_get());

void sensor_thread(void)
{
    while (1) {
        do_work();
        task_wdt_feed(chan); /* In the thread: feed regularly */
        k_msleep(500);
    }
}
```



Watchdogs configuration



```
# Hardware watchdog – last line of defence
CONFIG_WATCHDOG=y

# Per-thread software watchdog. Identifies WHICH thread is stuck
CONFIG_TASK_WDT=y
CONFIG_TASK_WDT_HW_FALLBACK=y    # use hardware WDT as backstop for task_wdt
```

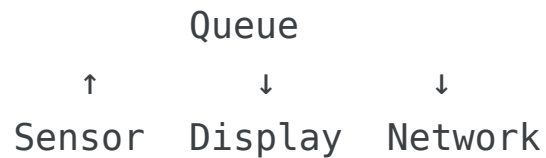
Layered watchdog strategy:

task_wdt (per-thread)	→	hw_wdt (whole system)
↓		↓
identifies stuck thread		resets if task_wdt itself hangs

- Feed `task_wdt` from every critical thread. The hardware watchdog is the safety net behind it 27



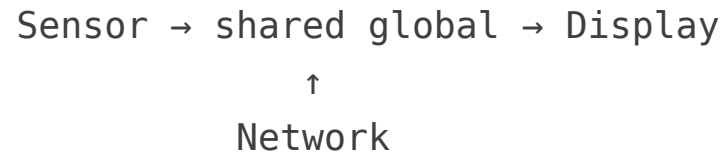
Good architecture:



Failure in sensor:

queue fills
display and network unaffected

Fragile architecture:



Failure in sensor:

global corrupted
display shows garbage
network sends bad data

- **Isolation boundary:** each subsystem owns its state
- **Communication boundary:** data crosses via well-defined channels
- **Error boundary:** failures stay inside their originating subsystem



Debugging & Detection Configuration



```
# Always enable during development
CONFIG_STACK_SENTINEL=y          # detect stack overflow at context switch
CONFIG_HW_STACK_PROTECTION=y     # detect overflow immediately (MPU required)
CONFIG_THREAD_ANALYZER=y         # measure stack high-water mark

# Fault output – essential for field diagnosis without a debugger
CONFIG_FAULT_DUMP=2              # full register dump (PC, LR, SP, fault regs)
CONFIG_EXTRA_EXCEPTION_INFO=y    # Cortex-M: exact fault address and status
```

- Full fault dumps turn "HardFault" into register and fault-address evidence
- Remove `CONFIG_THREAD_ANALYZER` from production, it adds runtime overhead



DEMO 2: Resource Budget Audit



Homework



- **Goal:** Study reliability under pressure
- **Task:**
 - Build a producer/consumer pipeline (from Lecture 4 homework)
 - Add a task watchdog
 - Simulate a stuck consumer (long `k_sleep` in consumer thread)
 - Observe: queue fills, watchdog fires, callback triggered
 - Add a health-check thread that monitors queue fill level and logs a warning at 75% capacity
- Push tag: `l5-task1`

Thank You





1. [Thread Stack APIs](#)
2. [Memory Heaps](#)
3. [Optimization Tools \(ram/rom report\)](#)
4. [Task Watchdog](#)
5. [Watchdog Driver API](#)
6. [Fatal Errors & Stack Detection](#)